

AI Usage Disclosure

AI Operations (AIOps) DA3408 — Module 3, Assignment 2

Infrastructure & Containerization: A Spam-Detection API

Roll No: DA24B025

1 Summary

A generative AI coding assistant was used while preparing this submission. This document lists every file whose initial content was drafted with that assistance, and states what was done with the rest of the submission. All AI-drafted files were read, edited where required, executed, and verified against the assignment's stated requirements before inclusion; every number reported in `report.pdf` and `README.md` is a measurement taken from actually running this code on my own machine and cluster, not a figure produced by the assistant.

2 Where AI was used

File(s)	Nature of the assistance
q1/train_model.py q2/train_model.py q4/train_model.py	Initial draft of the training script — the TF-IDF + MultinomialNB Pipeline, the <code>argparse</code> interface (<code>--data</code> , <code>--out</code>) and the <code>joblib</code> dump. The three copies are identical; each question folder keeps its own so that it remains a self-contained Docker build context.
q1/generate_dataset.py q2/generate_dataset.py q4/generate_dataset.py	Initial draft of the seeded synthetic dataset generator — the spam/ham message templates, the substitution vocabularies and the CSV writer. Also identical across the three folders.
q3/generate_shards.py	Initial draft of the synthetic signup-data generator — the record schema, the corruption modes used to seed malformed rows, and the seeded per-shard split into 8 CSV files with known invalid-row counts.
q3/validate_shard.py	Initial draft of the per-shard validator — the email/required-field validity rules, the shard selected by <code>JOB_COMPLETION_INDEX</code> , and the <code>RESULT_JSON</code> line emitted to stdout for later collection.
q3/collect_results.py	Initial draft of the results collector — listing the Job's pods by label selector and reading each pod's log through <code>read_namespaced_pod_log()</code> , then parsing the <code>RESULT_JSON</code> lines into the summary table. Adapted from the Lecture 6 exercise.
q2/docker-compose.yml q3/job.yaml q4/deployment.yaml q4/service.yaml	Initial drafts of the YAML manifests: the two-service Compose stack (<code>api</code> + <code>cache</code>), the Indexed Job, and the Deployment and NodePort Service.
README.md	Drafted with assistance, from the commands I ran and the outputs I observed.

3 What was decided and verified without AI

The parameters that the rubrics actually assess were chosen by me, from measurements taken on my own cluster and host, and were not taken from the assistant's drafts:

- the CPU budget analysis for Question 3 and the resulting `parallelism`, `completions` and `cpu` request/limit values in `q3/job.yaml`;
- the replica count, resource requests/limits, probe configuration and the rolling-update settings in `q4/deployment.yaml`, including the `preStop` hook added after observing dropped requests during a first rollout;
- all measured evidence — image sizes, cache hit/miss latencies, observed pod concurrency, per-shard invalid-row counts, self-healing timings and rolling-update request counts.

4 Files written without AI assistance

The remaining source in this repository was written by me: the API implementations (`q1/app.py`, `q2/app.py`, `q4/app.py`), all Dockerfiles and `.dockerignore` files, the pinned `requirements.txt` files, and `report.pdf`.

5 Responsibility

I have read and understood every line of the AI-assisted files listed above, and I take full responsibility for the correctness of the entire submission.